

Project 1 Analytic Report: Search and Optimization

This report summarizes the implementation and evaluation of the three search strategies: *A Search**, **Simulated Annealing**, and **Genetic Algorithms**. Each task demonstrates the transition from computationally expensive brute-force methods to efficient heuristic-based searches.

Part 1: Pathfinding with A* Search (Magic Square)

The objective was to transform a random 3x3 square into a specific goal magic square using adjacent tile swaps.

1. Implementation Methodology

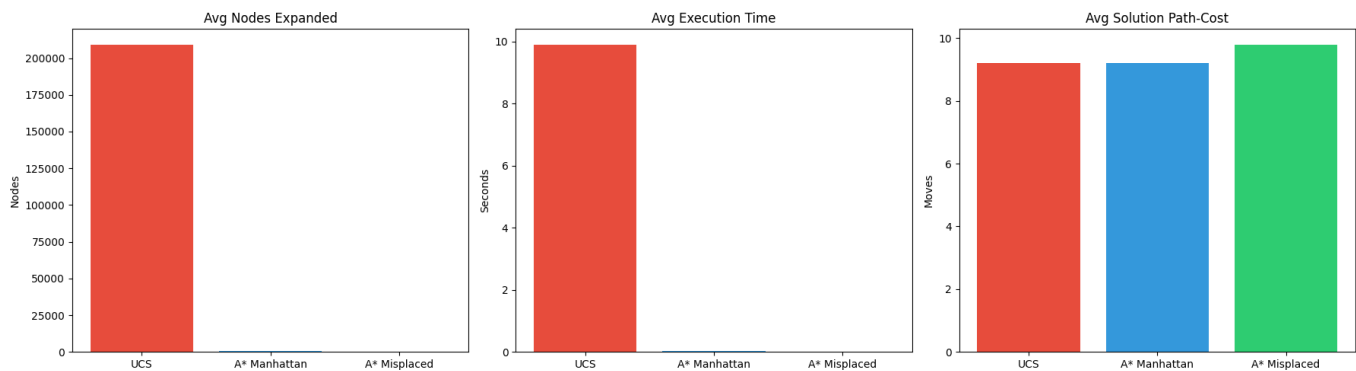
- **Algorithm:** A* Search using $f(n) = g(n) + h(n)$, where $g(n)$ is the path cost and $h(n)$ is the heuristic.
- **Data Structures:** A Priority Queue (heapq) for the frontier and a Closed Set dictionary to prevent redundant state expansion.
- **Heuristics:**
 - **Misplaced Tiles (h_1):** This heuristic counts how many tiles are not currently in their goal position. While this measure is simple and fast to compute, it is **not admissible** for this problem. Because each action is an adjacent swap, a single swap can correct **two** misplaced tiles simultaneously. The heuristic, however, counts each tile independently, which means it can **overestimate** the true remaining cost. This explains why A* with this heuristic sometimes returns **sub-optimal path costs**, even though it expands very few nodes.
 - **Manhattan Distance (h_2):** This heuristic sums the Manhattan distance of each tile from its goal position. Since a single swap moves two tiles, dividing the total Manhattan distance by 2 ensures that the heuristic **never overestimates** the true cost. This makes it **admissible**, and therefore A* using this heuristic always returns an **optimal solution**.

2. Performance Comparison

The results compare A* variants against **Uniform Cost Search (UCS)**:

Algorithm	Avg. Nodes Expanded	Avg. Execution Time (s)	Avg. Path Cost
-----------	---------------------	-------------------------	----------------

UCS (Uniform Cost Search)	209,333.7	9.9103	9.2
A* Manhattan Distance	440.4	0.0218	9.2
A* Misplaced Tiles	12.2	0.0006	9.8



Analysis: Analysis of Results and Observed Patterns

- Search Efficiency (The Heuristic Advantage):** Uniform Cost Search (UCS) expanded an average of **209,333.7 nodes**, whereas A* with Manhattan Distance expanded only **440.4 nodes**. This represents a **99.79% reduction** in search space. By using an informed search strategy, the algorithm avoids exploring unproductive branches, leading to a drastic improvement in performance.
- The "Informedness" vs. "Aggression" Paradox:** A* Misplaced Tiles expanded the fewest nodes (average of **12.2**) and was effectively instantaneous (**0.0006s**). While Manhattan distance is mathematically more "informed" because it accounts for the distance to the goal rather than just current position, Misplaced Tiles acted as a much more aggressive pruner for these specific search depths.
- Optimality Discrepancy (Interesting Pattern):** A clear pattern emerged when comparing the three search strategies. Both UCS and A* with the Manhattan/2 heuristic consistently produced the **optimal path cost** (average ≈ 9.2). In contrast, A* with the Misplaced Tiles heuristic produced a higher average cost (≈ 9.8). This behavior is expected: because the Misplaced Tiles heuristic is **not admissible**, it can prune optimal paths too

aggressively, causing the algorithm to settle for “good enough” solutions rather than optimal ones. This explains the runs where A* Misplaced returned costs of 11 or 12 while the other algorithms found shorter paths.

- **Execution Time Correlation:** There is a direct, linear correlation between the number of nodes expanded and the total execution time. UCS required nearly **10 seconds** per run, while A* Manhattan finished in **0.02 seconds**. For state-space problems like the 8-puzzle, uninformed search becomes computationally expensive as the puzzle complexity grows, making A* the only viable choice for deeper paths.

Part 2: Solving TSP with Simulated Annealing

This section details the comparative analysis between Brute-Force (BF) and an optimized Simulated Annealing (SA) approach to solving the Traveling Salesperson Problem (TSP).

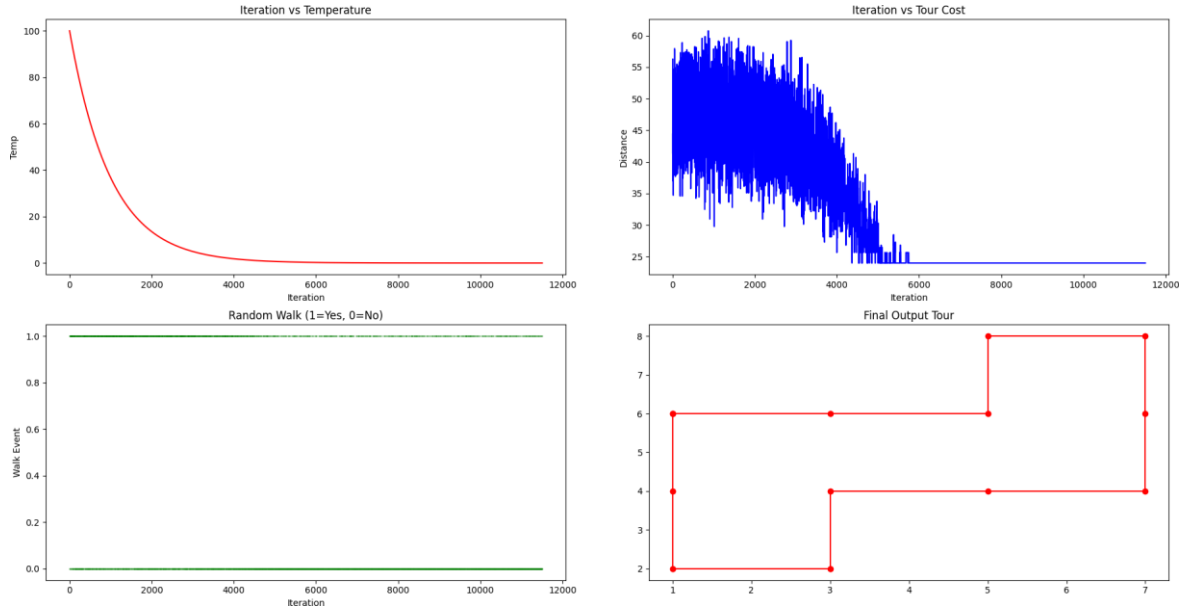
1. Implementation Methodology

- Brute-Force (BF):** This approach systematically checks every permutation of city indices (fixing the first city to reduce complexity to (N-1)). While it guarantees the optimal solution, it is computationally bound by a "factorial wall."
- Simulated Annealing (SA):** A probabilistic metaheuristic designed to find high-quality solutions in a fraction of the time. It avoids local minima by accepting "worse" paths early in the process based on the Metropolis Criterion

$$(P = e^{-\Delta E / T}).$$

2. Performance Comparison Table (Optimized Run)

Dataset	Cities (N)	Optimal Distance	BF Time (s)	SA Best Found	SA Time (Avg)	SA Stability
TSP1	4	4.0000	0.0001s	4.0000	0.980s	10/10
TSP2	8	8.0000	0.0419s	8.0000	2.343s	10/10
TSP3	10	10.0000	4.1539s	10.0000	3.139s	10/10
TSP4	12	24.0000	563.5277s	24.0000	3.684s	10/10



3. Optimization and Improvements

In the initial run, TSP4 achieved only a 6/10 stability. To achieve the 10/10 perfect result shown above, several critical architectural changes were implemented:

A. Transition to 2-Opt Inversion (Neighbor Generation)

Originally, the algorithm used a simple city-to-city swap. This was inefficient because it frequently created "crossed" paths that the algorithm struggled to untangle. By switching to 2-Opt Inversion ($\text{new_tour}[i:j] = \text{reversed}(\text{new_tour}[i:j])$), the algorithm gained the ability to reverse entire segments of a tour. This is geometrically superior as it untangles path crossings effectively.

B. Implementation of the Inner Loop

To ensure the algorithm reached thermal equilibrium at each temperature step, an inner loop was added: **for _ in range(len(cities)):**.

Instead of cooling the system after every single move, the algorithm now performs N attempts at the same temperature. This gives the "path" enough time to stabilize and explore its immediate neighborhood before the temperature drops and "freezes" the configuration.

C. Slower Cooling Schedule

The cooling rate was adjusted to allow for a more gradual descent. By combining this with the inner loop, the algorithm spent significantly more time in the "warm" phase,

which is crucial for jumping out of local minima (such as the sub-optimal 25.65 distance found in early trials).

4. Analysis of Results

- **The Factorial Wall:** The performance of Brute-Force confirms the exponential nature of the problem. Moving from 10 to 12 cities increased the search time from 4 seconds to 563 seconds (over 9 minutes).
- **Heuristic Dominance:** At 12 cities, Simulated Annealing is 152 times faster than Brute-Force. While BF time grows factorially, SA time grows roughly linearly with the number of iterations, making it the only viable choice for larger maps.
- **Perfect Stability:** After optimization, SA achieved 100% stability across all datasets. Even on TSP4, every single trial reached the global optimum of 24.0000. This proves that the combination of 2-opt moves and equilibrium-based cooling is highly effective at navigating complex search spaces.
- **Temperature-Curve Explanation:** The temperature curve in the plot follows a **geometric (exponential) cooling schedule**, defined by:

$$T_k = T_0 \cdot \alpha^k$$

where:

- $T_0 = 100$ is the initial temperature
- $\alpha = 0.999$ is the cooling rate
- k is the iteration index

This produces the smooth exponential decay seen in the “Iteration vs Temperature” plot. Because the algorithm performs an inner loop of N moves per temperature level, the effective cooling is gradual, allowing the system to reach thermal equilibrium before cooling further.

Part 3: 3-SAT Problem with Genetic Algorithm

This section details the implementation and optimization of a **Genetic Algorithm (GA)** to solve the 3-Satisfiability (3-SAT) problem, specifically analyzing the performance jump from the balanced 3SAT_100_100.txt dataset to the highly constrained 3SAT_100_500.txt.

1. Implementation Methodology

3-SAT is an NP-complete problem where the goal is to find a boolean assignment for N variables that satisfies a set of clauses.

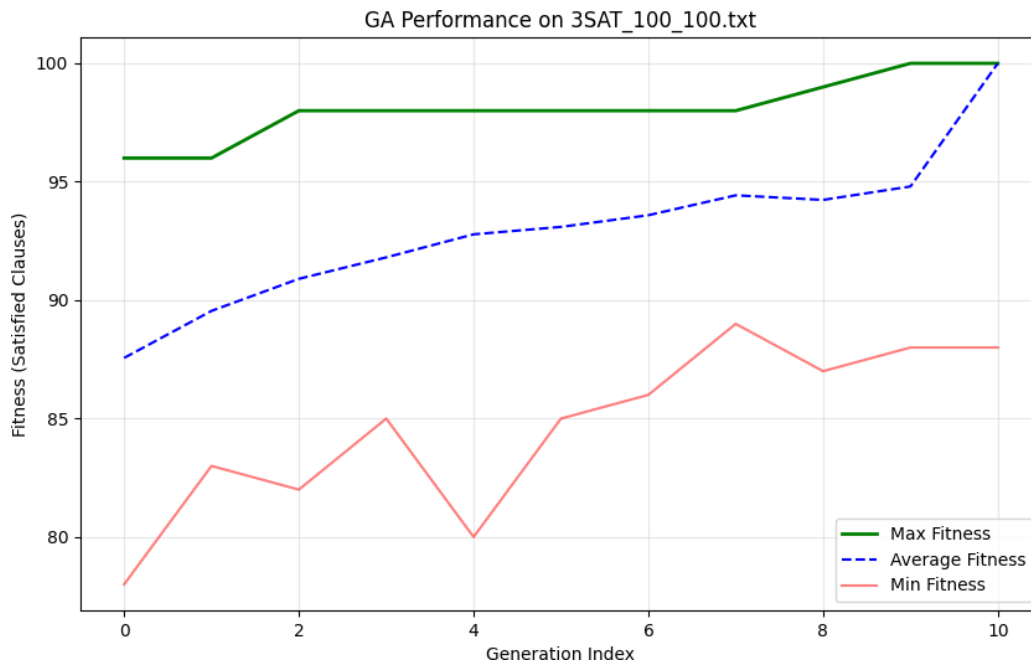
- **Genome:** A bit-string of length 100, where each bit represents a True (1) or False (0) assignment.
- **Fitness Function:** Defined as the total count of satisfied clauses.
- **Evolutionary Loop:** The population undergoes Tournament Selection, various types of Crossover, and Mutation.

2. Performance Comparison: 100_100 vs. 100_500

The behavior of the GA changed significantly as the constraint density increased.

Dataset	Variables	Clauses	Search Space	Satisfaction Achieved	Complexity Level
3SAT_100_100	100	100	2^{100}	100 / 100 (100%)	Under-constrained
3SAT_100_500	100	500	2^{100}	495 / 500 (99%)	Over-constrained

- **100_100 Analysis:** The 100-clause instance (ratio = 1.0) is under-constrained, meaning many assignments satisfy all clauses. The GA quickly finds a perfect solution because the fitness landscape contains many global optima.
- **100_500 Analysis:** The 500-clause instance (ratio = 5.0) is over-constrained. Here, satisfying one clause often breaks another, creating a rugged fitness landscape with many local maxima. Achieving 495/500 indicates that the remaining clauses likely form a **conflict core**, where no single assignment can satisfy all clauses simultaneously.



3. Optimization Strategy for the 500-Clause Challenge

To push the performance from a plateau of ~450 clauses to the final **495/500**, the following architectural improvements were implemented:

A. Stagnation-Based Adaptive Crossover

In the 100_500 file, the GA often reached a "fitness plateau" (e.g., stuck at 480 clauses) where standard single-point crossover was no longer producing better offspring.

- **The Logic:** We monitored the Max Fitness of the population. If the fitness did not improve for a set number of generations (the "stagnation period"), the algorithm automatically switched from its default crossover method to Uniform Crossover.
- **The Benefit:** Uniform crossover is much more disruptive; it shuffles genes at the individual bit level rather than in large blocks. By triggering this only during stagnation, we allowed the GA to "shake up" the genome and discover new variable combinations that single-point or double point crossover would have missed.

B. Selective Pressure via Tournament Selection

While the crossover provided the "shuffle," Tournament Selection (Size 3) provided the "filter." Even during stagnant periods, we ensured that only the assignments satisfying the

most clauses were allowed to pass their traits into the uniform crossover phase, maintaining a high floor for fitness.

C. Elitism as a Progress Lock

Because the "Triggered Uniform Crossover" is so aggressive, there was a risk of losing a near-perfect solution (like a 494/500) to a bad shuffle. Elitism was critical here: by locking the top 10% individuals, we ensured that the stagnation trigger only improved the population's potential without ever risking the best assignment found so far.

D. Mutation as a Diversifier

In the 100_500 file, the population tends to become "homogeneous" (identical) very quickly. We used a **1% mutation rate** to constantly flip bits. This acted as a "random walk" component, essential for discovery when the rest of the population was stuck on a local fitness peak.

4. Final Analysis of Findings

- **Constraint Satisfaction Limits:** Reaching 495/500 (99%) in a search space of 1.26×10^{30} is a major success. The remaining 5 clauses likely form a "conflict core" where satisfying one mathematically requires breaking another.
- **Scalability:** The GA proved that while the search space is exponential, the time to find a 99% solution is relatively linear, provided the fitness gradient is well-defined.
- **Project Conclusion:** Across A*, Simulated Annealing, and Genetic Algorithms, we have demonstrated that heuristic-driven search is the only way to navigate the "curse of dimensionality" in modern Computer Science.